

GCP Terraform + Ansible Lab (fixed HCL)

This replaces the files that failed terraform init. Terraform does not allow semicolons inside blocks. STEP 1 is mkdir. Every cat uses a full path. If a directory is missing, mkdir is in that same step.

HCL rule: one argument per line inside { }. Never write { family = "x"; project = "y" }.

STEP 1 — Create directories

```
mkdir -p /home/rps/tf-ansible-pg/env
mkdir -p /home/rps/tf-ansible-pg/modules/network
mkdir -p /home/rps/tf-ansible-pg/modules/iam
mkdir -p /home/rps/tf-ansible-pg/modules/compute
mkdir -p /home/rps/tf-ansible-pg/modules/sql
mkdir -p /home/rps/tf-ansible-pg/ansible/playbooks
mkdir -p /home/rps/tf-ansible-pg/ansible/inventories
ls -la /home/rps/tf-ansible-pg
```

STEP 2 — Enter directory

```
cd /home/rps/tf-ansible-pg
pwd
```

STEP 3 — Project id

```
gcloud config get-value project
```

STEP 4 — tfvars

```
mkdir -p /home/rps/tf-ansible-pg/env
PROJECT_ID=$(gcloud config get-value project)
echo "PROJECT_ID is $PROJECT_ID"
cat > /home/rps/tf-ansible-pg/env/terraform.tfvars <<TFVARS
project_id      = "${PROJECT_ID}"
region          = "asia-south1"
zone            = "asia-south1-a"
network_name    = "pg-lab-vpc"
subnet_name     = "pg-lab-subnet"
subnet_cidr     = "10.20.0.0/24"
vm_name         = "pg-ansible-vm"
sql_name        = "pg-lab-sql"
sql_password    = "ChangeMe-Lab-Only"
create_network  = false
TFVARS
cat /home/rps/tf-ansible-pg/env/terraform.tfvars
```

STEP 5 — APIs

```
gcloud services enable compute.googleapis.com sqladmin.googleapis.com iam.googleapis.com servicenetworking.googleapis.com cloudresourcemanager.v1
```

STEP 6 — VPC and subnet (1.1) if missing

```
gcloud compute networks describe pg-lab-vpc || gcloud compute networks create pg-lab-vpc --subnet-mode=custom --bgp-routing-mode=regional
gcloud compute networks subnets describe pg-lab-subnet --region=asia-south1 || gcloud compute networks subnets create pg-lab-subnet --network=pg-lab-vpc
```

STEP 7 — Firewalls (1.2) if missing

```
gcloud compute firewall-rules describe pg-lab-allow-iap-ssh || gcloud compute firewall-rules create pg-lab-allow-iap-ssh --network=pg-lab-vpc --source-ranges=0.0.0.0/0
gcloud compute firewall-rules describe pg-lab-allow-5432-internal || gcloud compute firewall-rules create pg-lab-allow-5432-internal --network=pg-lab-vpc
```

STEP 8 — PSA (1.3) if missing

```
gcloud compute addresses describe pg-lab-psa --global || gcloud compute addresses create pg-lab-psa --global --purpose=VPC_PEERING --prefix-length=24
gcloud services vpc-peerings connect --service=servicenetworking.googleapis.com --ranges=pg-lab-psa --network=pg-lab-vpc || true
gcloud services vpc-peerings list --network=pg-lab-vpc
```

STEP 9 — versions.tf

```
mkdir -p /home/rps/tf-ansible-pg
cat > /home/rps/tf-ansible-pg/versions.tf <<'EOF'
terraform {
  required_version = ">= 1.6.0"
  required_providers {
    google = {
      source = "hashicorp/google"
      version = "> 5.0"
    }
  }
}
```

```

}

provider "google" {
  project = var.project_id
  region  = var.region
}
EOF

```

STEP 10 — variables.tf (multiline, no semicolons)

```

cat > /home/rps/tf-ansible-pg/variables.tf <<'EOF'
variable "project_id" {
  type = string
}

variable "region" {
  type = string
}

variable "zone" {
  type = string
}

variable "network_name" {
  type = string
}

variable "subnet_name" {
  type = string
}

variable "subnet_cidr" {
  type = string
}

variable "vm_name" {
  type = string
}

variable "sql_name" {
  type = string
}

variable "sql_password" {
  type      = string
  sensitive = true
}

variable "create_network" {
  type      = bool
  default   = false
}
EOF

```

STEP 11 — modules/network/main.tf (fixed)

```

mkdir -p /home/rps/tf-ansible-pg/modules/network
cat > /home/rps/tf-ansible-pg/modules/network/main.tf <<'EOF'
variable "create_network" {
  type = bool
}

variable "network_name" {
  type = string
}

variable "subnet_name" {
  type = string
}

variable "subnet_cidr" {
  type = string
}

variable "region" {
  type = string
}

data "google_compute_network" "this" {
  name = var.network_name
}

data "google_compute_subnetwork" "this" {
  name      = var.subnet_name
  region    = var.region
}

```

```

resource "google_compute_global_address" "psa" {
  name           = "pg-lab-psa"
  purpose        = "VPC_PEERING"
  address_type   = "INTERNAL"
  prefix_length  = 16
  network        = data.google_compute_network.this.id
}

resource "google_service_networking_connection" "psa" {
  network            = data.google_compute_network.this.id
  service            = "servicenetworking.googleapis.com"
  reserved_peering_ranges = [google_compute_global_address.psa.name]
}

resource "google_compute_firewall" "iap_ssh" {
  name    = "pg-lab-allow-iap-ssh"
  network = data.google_compute_network.this.name

  allow {
    protocol = "tcp"
    ports    = ["22"]
  }

  source_ranges = ["35.235.240.0/20"]
  target_tags   = ["pg-lab"]
}

resource "google_compute_firewall" "pg" {
  name    = "pg-lab-allow-5432-internal"
  network = data.google_compute_network.this.name

  allow {
    protocol = "tcp"
    ports    = ["5432"]
  }

  source_ranges = [var.subnet_cidr]
  target_tags   = ["pg-lab"]
}

output "network_id" {
  value = data.google_compute_network.this.id
}

output "subnet_self_link" {
  value = data.google_compute_subnetwork.this.self_link
}
EOF

```

STEP 12 — modules/iam/main.tf

```

mkdir -p /home/rps/tf-ansible-pg/modules/iam
cat > /home/rps/tf-ansible-pg/modules/iam/main.tf <<'EOF'
variable "project_id" {
  type = string
}

resource "google_service_account" "vm" {
  account_id   = "pg-vm-sa"
  display_name = "PostgreSQL VM"
}

resource "google_service_account" "sql" {
  account_id   = "pg-sql-client-sa"
  display_name = "Cloud SQL client"
}

resource "google_project_iam_member" "vm_log" {
  project = var.project_id
  role    = "roles/logging.logWriter"
  member  = "serviceAccount:${google_service_account.vm.email}"
}

resource "google_project_iam_member" "vm_mon" {
  project = var.project_id
  role    = "roles/monitoring.metricWriter"
  member  = "serviceAccount:${google_service_account.vm.email}"
}

resource "google_project_iam_member" "sql_client" {
  project = var.project_id
  role    = "roles/cloudsql.client"
  member  = "serviceAccount:${google_service_account.sql.email}"
}

output "vm_email" {
  value = google_service_account.vm.email
}

```

```

}

output "sql_email" {
  value = google_service_account.sql.email
}
EOF

```

STEP 13 — modules/compute/main.tf (fixed)

```

mkdir -p /home/rps/tf-ansible-pg/modules/compute
cat > /home/rps/tf-ansible-pg/modules/compute/main.tf <<'EOF'
variable "name" {
  type = string
}

variable "zone" {
  type = string
}

variable "subnet" {
  type = string
}

variable "sa_email" {
  type = string
}

data "google_compute_image" "ubuntu" {
  family  = "ubuntu-2404-lts-amd64"
  project = "ubuntu-os-cloud"
}

resource "google_compute_instance" "vm" {
  name          = var.name
  machine_type  = "e2-medium"
  zone         = var.zone
  tags          = ["pg-lab"]

  boot_disk {
    initialize_params {
      image = data.google_compute_image.ubuntu.self_link
      size  = 20
      type  = "pd-balanced"
    }
  }

  network_interface {
    subnetwork = var.subnet
  }

  service_account {
    email  = var.sa_email
    scopes = ["cloud-platform"]
  }

  metadata = {
    enable-oslogin = "TRUE"
  }
}

output "internal_ip" {
  value = google_compute_instance.vm.network_interface[0].network_ip
}
EOF

```

STEP 14 — modules/sql/main.tf (fixed)

```

mkdir -p /home/rps/tf-ansible-pg/modules/sql
cat > /home/rps/tf-ansible-pg/modules/sql/main.tf <<'EOF'
variable "name" {
  type = string
}

variable "region" {
  type = string
}

variable "network_id" {
  type = string
}

variable "password" {
  type          = string
  sensitive     = true
}

```

```

resource "google_sql_database_instance" "this" {
  name             = var.name
  database_version = "POSTGRES_16"
  region           = var.region

  settings {
    tier                = "db-custom-1-3840"
    availability_type = "ZONAL"
    disk_size          = 20

    ip_configuration {
      ipv4_enabled = false
      private_network = var.network_id
    }
  }
}

root_password = var.password

resource "google_sql_database" "appdb" {
  name     = "appdb"
  instance = google_sql_database_instance.this.name
}

resource "google_sql_user" "appuser" {
  name       = "appuser"
  instance   = google_sql_database_instance.this.name
  password   = var.password
}

output "connection_name" {
  value = google_sql_database_instance.this.connection_name
}

output "private_ip" {
  value = google_sql_database_instance.this.private_ip_address
}
EOF

```

STEP 15 — main.tf

```

cat > /home/rps/tf-ansible-pg/main.tf <<'EOF'
module "network" {
  source      = "../modules/network"
  create_network = var.create_network
  network_name = var.network_name
  subnet_name  = var.subnet_name
  subnet_cidr  = var.subnet_cidr
  region       = var.region
}

module "iam" {
  source      = "../modules/iam"
  project_id  = var.project_id
}

module "compute" {
  source      = "../modules/compute"
  name        = var.vm_name
  zone        = var.zone
  subnet      = module.network.subnet_self_link
  sa_email    = module.iam.vm_email
}

module "sql" {
  source      = "../modules/sql"
  name        = var.sql_name
  region      = var.region
  network_id  = module.network.network_id
  password    = var.sql_password
  depends_on  = [module.network]
}

output "vm_ip" {
  value = module.compute.internal_ip
}

output "sql_connection" {
  value = module.sql.connection_name
}

output "sql_private_ip" {
  value = module.sql.private_ip
}
EOF

```

STEP 16 — Format and init (must be clean)

```
cd /home/rps/tf-ansible-pg
terraform fmt -recursive
terraform init
```

If fmt or init still reports Invalid character, a file still has a semicolon. Re-run the cat for that file from this PDF.

STEP 17 — Import existing 1.1–1.3 objects

```
cd /home/rps/tf-ansible-pg
PROJECT_ID=$(gcloud config get-value project)
terraform import -var-file=/home/rps/tf-ansible-pg/env/terraform.tfvars module.network.google_compute_global_address.psa pg-lab-psa || true
terraform import -var-file=/home/rps/tf-ansible-pg/env/terraform.tfvars module.network.google_compute_firewall.iap_ssh pg-lab-allow-iap-ssh || true
terraform import -var-file=/home/rps/tf-ansible-pg/env/terraform.tfvars module.network.google_compute_firewall.pg pg-lab-allow-5432-internal || true
terraform import -var-file=/home/rps/tf-ansible-pg/env/terraform.tfvars 'module.network.google_service_networking_connection.psa' "projects/${PROJECT_ID}/global/serviceNetworkingConnections/pg-lab-allow-5432-internal"
```

STEP 18 — Plan and apply

```
cd /home/rps/tf-ansible-pg
terraform plan -var-file=/home/rps/tf-ansible-pg/env/terraform.tfvars -out=tfplan
terraform apply tfplan
terraform output
```

STEP 19–20 — Ansible on the VM

```
gcloud compute ssh pg-ansible-vm --zone=asia-south1-a --tunnel-through-iap
```

On the VM:

```
sudo apt-get update
sudo apt-get install -y ansible python3-psycopg2 postgresql-16 postgresql-contrib-16 || sudo apt-get install -y ansible python3-psycopg2
ansible-galaxy collection install community.postgresql
```

Create postgres.yml on the VM (same playbook as before: apt postgresql-16, listen_addresses, pg_hba, appuser, appdb, backup dir) then:

```
ansible-playbook -i localhost, -c local postgres.yml
sudo -u postgres psql -c "SELECT version();"
sudo -u postgres pg_dump -Fc appdb -f /var/backups/postgresql/appdb.dump
```

STEP 21 — Destroy when finished

```
cd /home/rps/tf-ansible-pg
terraform destroy -var-file=/home/rps/tf-ansible-pg/env/terraform.tfvars
```

What this PDF fixes vs the last init error

compute data image and boot_disk are multiline. network subnet data is multiline. sql variable password is two lines (type + sensitive). No ; inside any { }.